SUP DE VINCI

26 Février 2026

Projet d'études M1 DATA

NLP sur Bluesky avec Kedro

MASTÈRE BIG DATA & IA

FORMATEUR : DIALLO ALIMOU

Pipeline complet :

Posts Bluesky

↓

Nettoyage NLP

↓

Vectorisation

↓

Modèle ML / IA

C’est quoi la vectorisation

La **vectorisation** consiste à transformer du texte (mots, phrases, documents) en vecteurs numériques.
Ces vecteurs sont des représentations mathématiques que les algorithmes peuvent analyser.

Exemple concret (mairie)

Un citoyen écrit :
"Problème avec l’éclairage public dans ma rue" [0.12, 0.98, 0.03, 0.44, ...]

Grâce à ça, le système peut :

- Trier automatiquement les messages
- Classer en catégories :
  - État civil
  - Infrastructure
  - Services publics

L’amélioration par la pondération TF-IDF

Pour pallier l’un des défauts du modèle BoW – le fait que des mots très fréquents mais peu informatifs (comme « le », « de », « un ») dominent le vecteur –, la méthode TF-IDF (Term Frequency-Inverse Document Frequency) a été développée. Elle affine la représentation en pondérant l’importance de chaque mot.

❓ Problème

Tous les mots n’ont pas la même importance :

Mots comme :

"le", "de", "et", "est", "un"

apparaissent partout → peu informatifs

Mots comme :

"covid", "élection", "incendie", "grève"

rares mais très importants

L’avènement des « Word Embeddings » (Word2Vec, GloVe)

Les « word embeddings » (plongements de mots) sont la première génération de modèles neuronaux dédiés à la vectorisation des données. Des algorithmes comme Word2Vec et GloVe ont révolutionné le Traitement Automatique du Langage (NLP). Contrairement aux vecteurs creux et de très grande dimension de BoW/TF-IDF, les word embeddings sont des vecteurs denses, de plus petite dimension (typiquemen entre 50 et 300).

Avec Word2Vec

vecteur("roi") – vecteur("homme") + vecteur("femme") ≈ vecteur("reine")

Les représentations contextuelles (BERT & Transformers)

Malgré leur puissance, les word embeddings classiques comme Word2Vec ont une limite majeure : ils assignent un seul et unique vecteur à chaque mot, quel que soit son contexte. Or, de nombreux mots sont polysémiques. Le mot « avocat » n'a pas le même sens dans « **l'avocat défend son client** » et « **j'aime la salade d'avocat** ».

La révolution suivante est venue avec les modèles de langage basés sur l'architecture Transformer, dont **BERT** (Bidirectional Encoder Representations from Transformers) est l'un des pionniers. L'innovation fondamentale de ces modèles est de générer des représentations contextuelles. Le vecteur d'un mot est calculé dynamiquement en fonction de la phrase entière dans laquelle il se trouve, grâce à un mécanisme d'attention qui pondère l'influence des autres mots.
Ainsi, le mot « **avocat** » aura deux vecteurs très différents dans les exemples précédents, résolvant ainsi le problème de la polysémie.

Comprendre la vectorisation avec des exemples simples

Exemple 1 — Phrases simples (Bag of Words)

1. "J'aime le football"
2. "J'aime le tennis"
3. "Le football est populaire"

Vocabulaire global ["j'aime", "le", "football", "tennis", "est", "populaire"]

Représentation vectorielle

```
Phrase 1 → [1, 1, 1, 0, 0, 0]
Phrase 2 → [1, 1, 0, 1, 0, 0]
Phrase 3 → [0, 1, 1, 0, 1, 1]
```

Exemple 1 — Phrases simples (Bag of Words)

```
hp@santtoudllo MINGW64 ~/Desktop/sup de vinci/Supdevinci-2025-2026/m1/scr (main)
$ python ./demo_vectorisation_mot.py
['aime' 'est' 'football' 'le' 'populaire' 'tennis']
[[0.61980538 0.         0.61980538 0.48133417 0.         0.        ]
 [0.54783215 0.         0.         0.42544054 0.         0.72033345]
 [0.         0.5844829  0.44451431 0.34520502 0.5844829  0.        ]]
```

Ligne 1 : "j aime le football" [0.6198, 0, 0.6198, 0.4813, 0, 0]

Un autre exemple avec les données Bluesky

```
docs = [
    "breaking news election",
    "football match tonight",
    "election results today",
]
```

```
$ python vectorisation_bluesky.py
Vocabulaire : ['breaking' 'election' 'football' 'match' 'news' 'results' 'today'
 'tonight']

Matrice TF-IDF :

   breaking  election  football    match      news   results     today  tonight
0  0.622766   0.47363   0.00000  0.00000  0.622766  0.000000  0.000000  0.00000
1  0.000000   0.00000   0.57735  0.57735  0.000000  0.000000  0.000000  0.57735
2  0.000000   0.47363   0.00000  0.00000  0.000000  0.622766  0.622766  0.00000
```

TF-IDF transforme chaque post en vecteur numérique.

Les mots rares mais informatifs ont un **poids fort**, les mots communs ont un **poids faible**.

Lien direct avec mon pipeline Kedro

Dans ton pipeline :

Input : cleaned_posts.pkl

```
docs = [p["clean_text"] for p in cleaned_posts]
vectorizer.fit_transform(docs)
```

Modèle ML / IA

On a transformé du texte en vecteurs numériques (TF-IDF).

Maintenant, on va utiliser des algorithmes de Machine Learning pour :

- Regrouper les posts par thème (clustering)
- Classifier les posts (classification supervisée)
- Détecter des tendances
- Plus tard : fake news, sujets sensibles, etc.

Un modèle de Machine Learning ne comprend PAS le texte Il ne comprend que des nombres (vecteurs)

Clustering — Regrouper automatiquement les posts

Le clustering est une technique de Machine Learning qui consiste à regrouper automatiquement des données similaires entre elles, sans connaître les catégories à l’avance.

C’est de l’apprentissage non supervisé :

- On ne donne pas de labels
- Le modèle découvre tout seul les groupes (clusters)

Le clustering, c’est comme trier des objets sans savoir à l’avance leurs catégories, uniquement en regardant leurs ressemblances.

Clustering — Regrouper automatiquement les posts

Exemple très concret (vie réelle)

Trier des chaussettes

Tu as un tas de chaussettes mélangées :

- Le clustering va faire des piles :
  - Couleurs similaires
  - Tailles similaires
  - Types similaires

Tu n’as pas dit au modèle :

"Ceci est une chaussette rouge"

Il a juste regroupé ce qui se ressemble

Classification — Trier les posts par catégorie

Cette fois, on donne des exemples déjà classés au modèle.

Exemples d’apprentissage :

Texte	Label
"election results"	politique
"football match"	sport

Algorithme classique K-Means

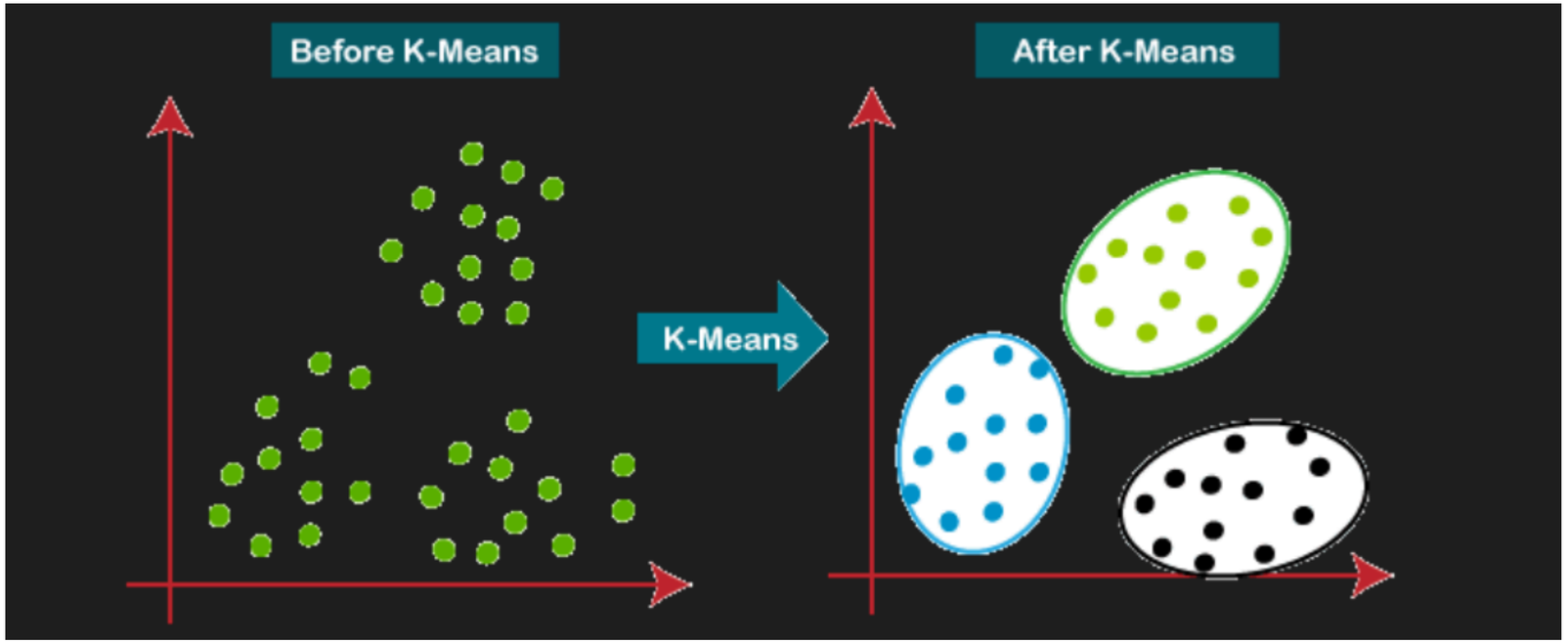
L’algorithme de clustering le plus simple et le plus utilisé.

Il :

Choisit K groupes

Place des centres

Regroupe les points les plus proches



Après la création du modèle : et maintenant ?

Nous avons :

- Collecté les données
- Nettoyé les posts
- Vectorisé (TF-IDF)
- Entraîné un modèle (KMeans / Classifier)

Question :

Comment rendre ce modèle utilisable dans la vraie vie ?

Sauvegarde du modèle (joblib)

```
import joblib
joblib.dump(model, "data/06_models/model.joblib")
joblib.dump(vectorizer, "data/06_models/vectorizer.joblib")
```

Ce qu’il faut TOUJOURS sauvegarder

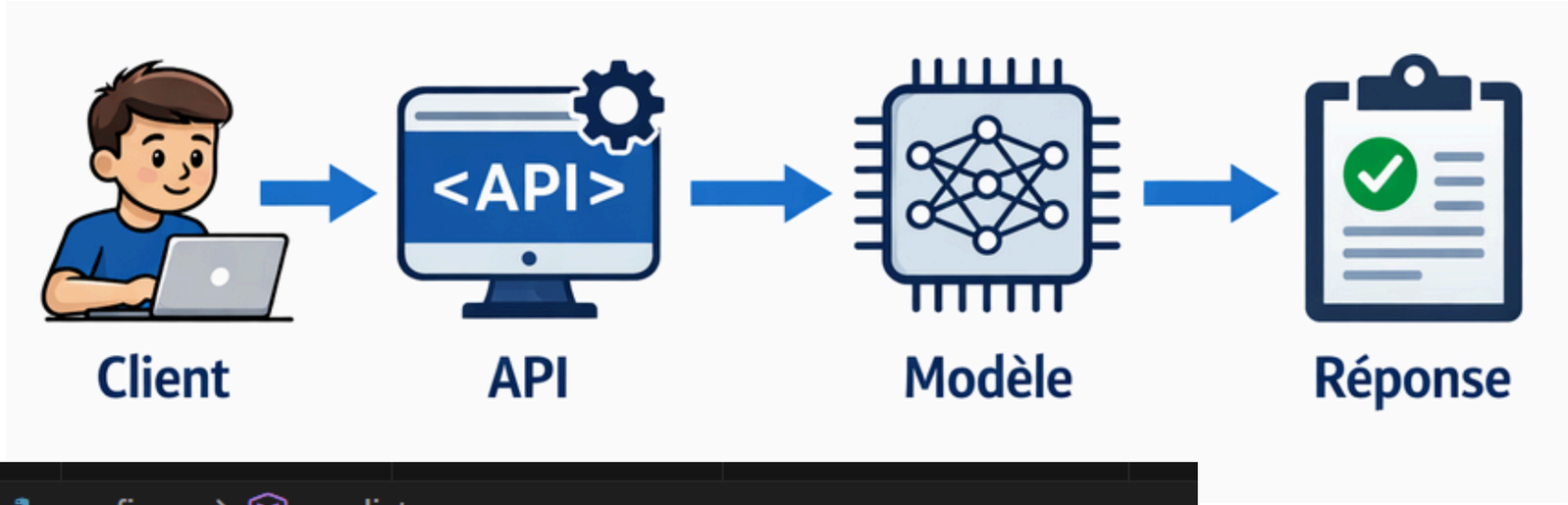
Élément	Pourquoi
vectorizer	Transformer les nouveaux textes
model	Faire la prédiction
label encoder (si classification)	Traduire les classes

créer une API

Parce que le modèle doit être accessible :

- Par un site web
- Par un dashboard
- Par un autre système
- En temps réel

API avec FastAPI



src > config.py > predict

```python
from fastapi import FastAPI
import joblib

app = FastAPI()

model = joblib.load("model.joblib")
vectorizer = joblib.load("vectorizer.joblib")

@app.post("/predict")
def predict(text: str):
    X = vectorizer.transform([text])
    prediction = model.predict(X)
    return {"prediction": int(prediction[0])}
```

Fonctionnement de l’API

L’utilisateur envoie un texte

- **L’API :**
- transforme en vecteur
- applique le modèle
- **L’API retourne :**
- cluster
- catégorie
- score

Exemple Streamlit

```python
import streamlit as st
import requests

text = st.text_input("Entrer un post")

if st.button("Analyser"):
    r = requests.post("http://localhost:8000/predict", json={"text": text})
    st.write(r.json())
```